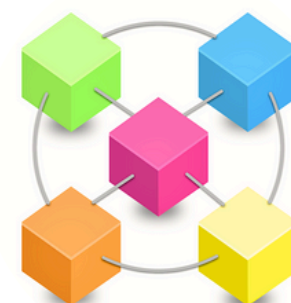
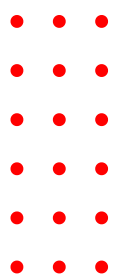
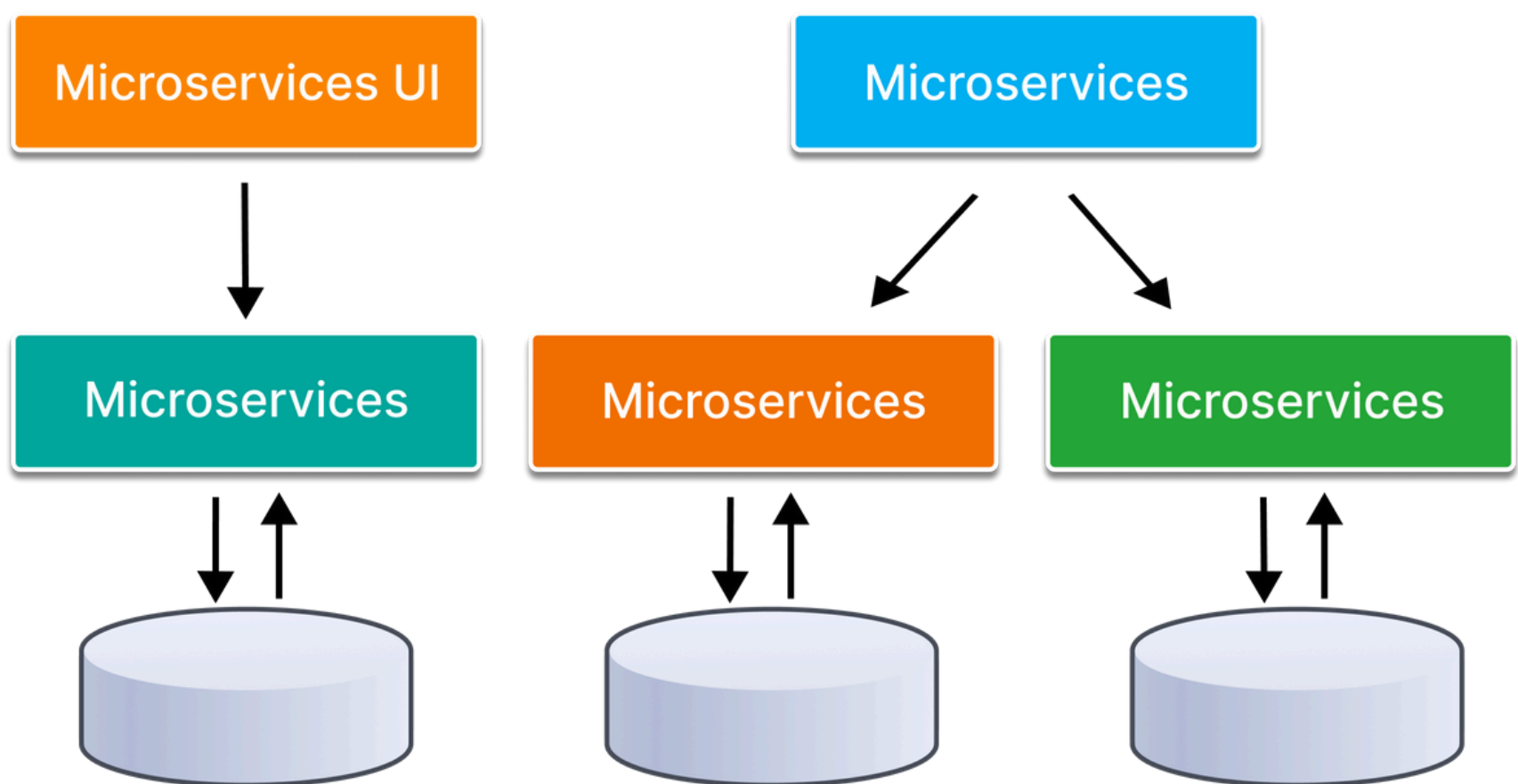




Spring Boot *and* Microservices



In the modern era, building fast, agile, and scalable systems is a crucial requirement for every technological solution. **Microservice architecture** is an excellent approach to meet these demands. Spring Boot is recognized as one of the most powerful and straightforward platforms for microservices.



Our **Auto API Generator plugin** automatically creates RESTful APIs in accordance with best practices based on your entity classes in just a few seconds.

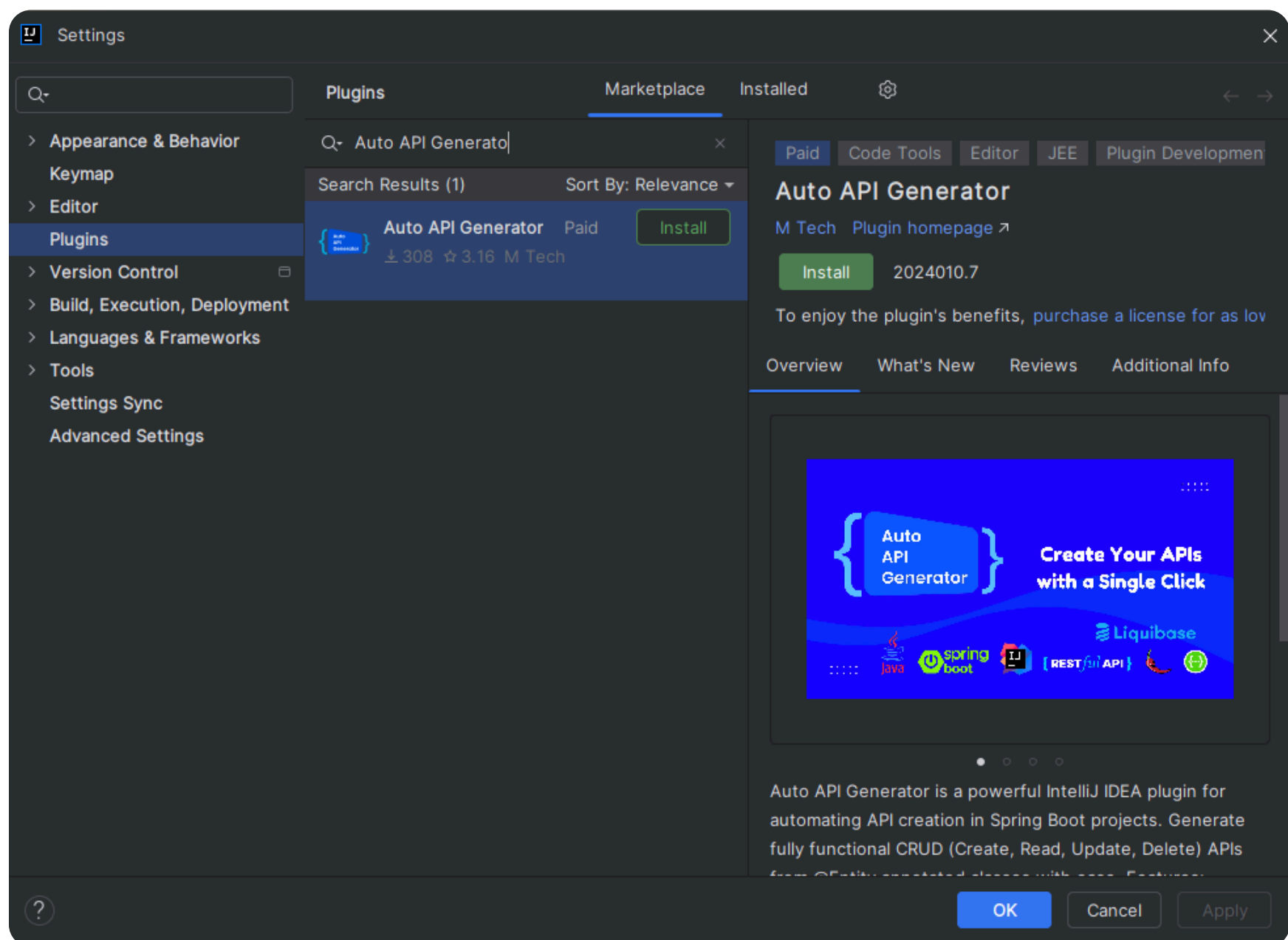
Create the complete **API structure (DTO, Mapper, Repository, Service, Controller)** with just a few clicks!

Speed up your work with **Lombok, Liquibase, File Upload API**, and **Exception Handling!**

And much more— all the features you need in one place!

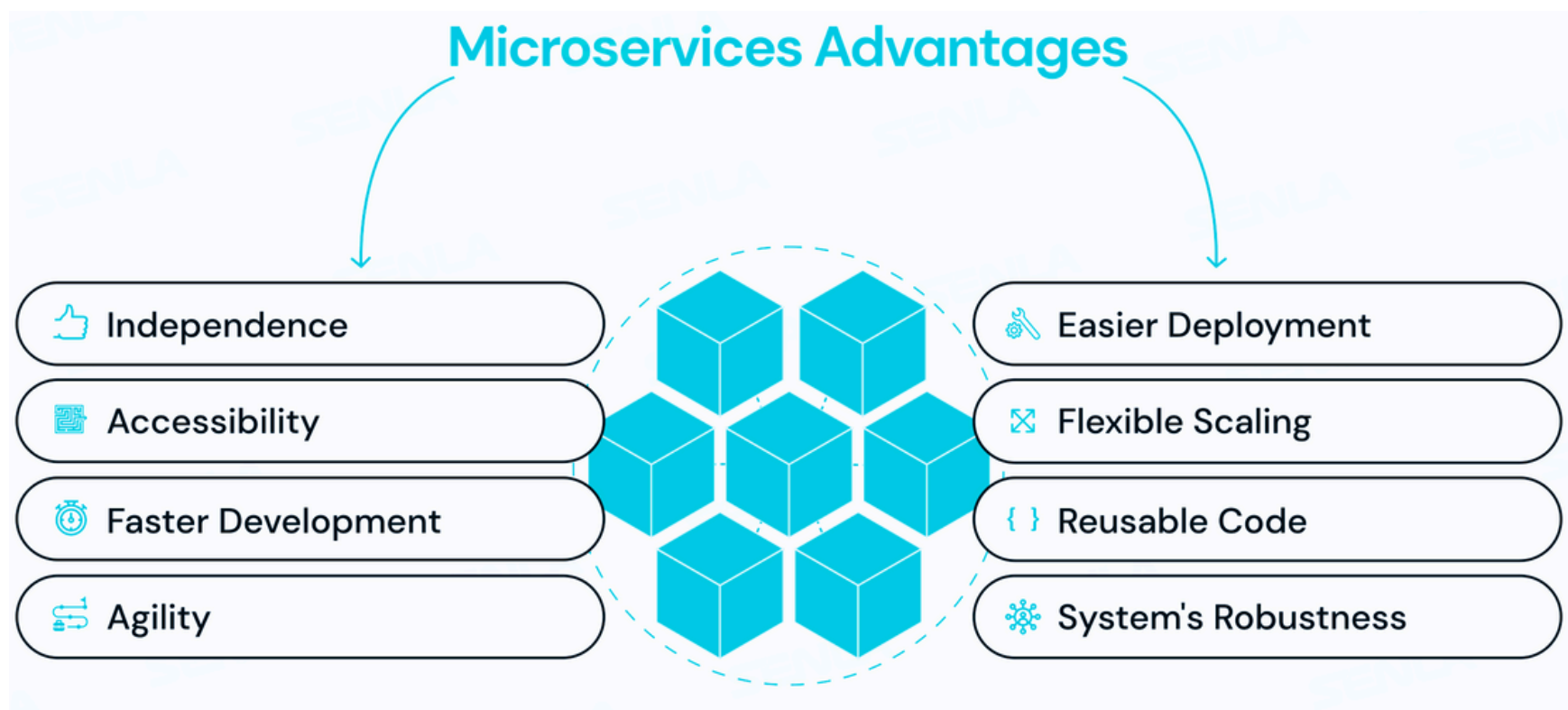
Installing the Auto API Generator Plugin:

1. In IntelliJ IDEA, go to the Plugins section.
2. Open the Marketplace tab and search for Auto API Generator.
3. Click the Install button.



Core Principles of the Microservices Approach

The microservices approach is used to break down large and complex applications into smaller, independent services. Each of these services performs its task correctly and independently. Below, I have listed the fundamental rules for building a perfect system by adhering to microservices principles:



Independence Between Services: Each microservice must operate independently. Any issue in one service should not affect other services. This ensures the proper functioning of the services and the reliability of the system.

Instance Independence: Each service's instances should only work with their own version. For example, if one instance works with *v1*, another should not work with *v2*. This simplifies version management for services and minimizes disruptions during updates.

Each Service Has Its Own Domain: Each microservice should only perform its relevant task. For instance, a User Service should only handle user registration. If we start handling image operations within this service, we violate microservices principles. Each service should focus solely on its domain and not include other functionalities.

Use of Suitable Technologies: Each service should be built with the technology that best suits its functional requirements. For example, a database for user management and specific tools for authentication can be used, but a different technology should be chosen for image operations.

Container and Database Isolation: Each service should run in its own independent container, and each service should have its own dedicated database. This ensures independent deployment and management of services.

CI/CD and Automated Deployment: Each service should have its own CI/CD (Continuous Integration / Continuous Deployment) processes. Any changes made to a service or functionality should be deployed independently through separate deployment operations. This increases agility in the development process and ensures continuous service delivery.

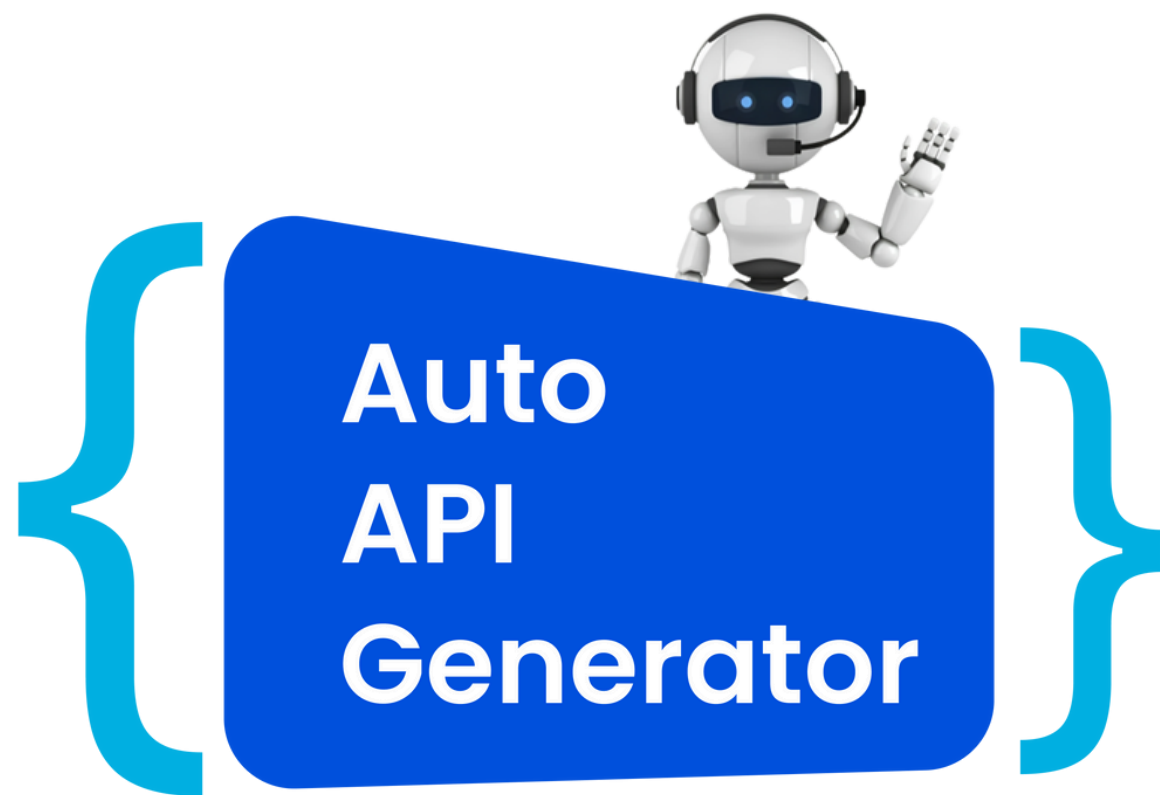
Adherence to Relationships and Rules: The relationships between services should be predefined and should not deviate from these relationships. Each service should interact with other services based solely on its respective rules and protocols. This approach ensures independence and scalability within the system.

Logging and Security Management: Each service should independently implement logging, security, and other management practices. Logs should help track the performance of each service, and security protocols must ensure the overall security of the system. Additionally, each service should properly manage access control.

To learn more and see a real project, you can check out the following link:

<https://github.com/musanabiyev/e-commerce-microservices>.





**More functionality will be added soon.
Save time, speed up your work, and make
your life easier with the Auto API
Generator plugin!**